МИНОБРНАУКИ РОССИИ
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«МИРЭА – Российский технологический университет»
РТУ МИРЭА

Практическое занятие № 4

Тема занятия: эффективная работа с функциями puzzle rpa
Вариант: 11

Выполнил студент 3 курса группы
ЭФБО-09-23 Калинин Н.В.

Проверил Громов Е.В.

Москва
2026 г.

Цель работы: сформировать у студентов понимание функционального подхода в разработке RPA-процессов и научить применять функции в Puzzle RPA для оптимизации архитектуры робота, повторного использования логики и повышения читаемости алгоритма.

ТЕОРЕТИЧЕСКАЯ ЧАСТЬ

Современные системы RPA позволяют создавать программных роботов для выполнения повторяющихся бизнес-операций. Однако по мере усложнения задач возникает необходимость структурирования алгоритмов, разделения логики и повышения управляемости процесса.

Одним из ключевых инструментов структурирования является использование функций.

Изучение функций в Puzzle RPA – это момент, когда вы переходите от простого создания роботов к настоящему проектированию автоматизированных решений.

До этого этапа алгоритмы могли выглядеть как последовательность действий. После освоения функций вы начнёте мыслить как разработчик: структурировать процесс, продумывать архитектуру и создавать универсальные решения,

которые можно масштабировать и совершенствовать.

Функции открывают новые возможности:

- вы научитесь разбивать сложную задачу на понятные управляемые части;
- сможете создавать «умные» модули, которые работают независимо;
- перестанете дублировать действия и упростите логику процессов;
- сможете создавать решения, которые выглядят профессионально и уверенно защищаются.

Это шаг от учебного алгоритма к инженерному подходу.

После изучения темы вы сможете:

- строить архитектуру робота осознанно;
- создавать масштабируемые процессы;
- легко вносить изменения в проекты;
- автоматизировать более сложные бизнес-сценарии;
- чувствовать уверенность при защите своей работы.

Функции – это инструмент, который превращает исполнителя алгоритма в проектировщика системы.

Именно здесь начинается уровень профессиональной разработки в RPA.

Понятие функции в RPA

В RPA функция – это самостоятельный логический блок алгоритма, предназначенный для выполнения конкретной задачи в рамках общего процесса.

Функция позволяет выделить часть действий в отдельный модуль, который можно:

- вызывать при необходимости;
- использовать повторно;
- изменять независимо от основного процесса;
- тестировать отдельно.

Проще говоря, функция – это «отдельный механизм» внутри робота, выполняющий одну чётко определённую операцию.

В RPA-решениях процессы часто включают:

- чтение данных;
- проверку условий;
- обработку информации;
- формирование отчётов;
- запись результатов.

Если вся логика записана последовательно в одном блоке, алгоритм становится громоздким и сложным для понимания.

Функции позволяют:

- структурировать процесс;
- разделить ответственность между модулями;
- упростить сопровождение проекта;
- повысить устойчивость робота к изменениям.

Таким образом, функция является инструментом архитектурного построения автоматизированного решения.

Основные характеристики функции в RPA

Функция в роботизированной автоматизации процессов – это не просто удобный инструмент, а фундаментальный элемент архитектуры. Её характеристики определяют устойчивость, гибкость и профессиональный уровень разработки.

Рассмотрим ключевые свойства функции подробно.

1. *Самостоятельность* (Завершённость логики)

Каждая функция должна выполнять одну конкретную задачу и доводить её до логического завершения.

Принцип: «Одна функция – одна ответственность».

Например:

- функция проверки листа – только проверяет;
- функция обработки строки – только обрабатывает строку;
- функция записи – только записывает данные.

Принцип единственной ответственности (Single Responsibility Principle) является одним из базовых принципов качественной архитектуры. Его нарушение приводит к усложнению поддержки и росту количества ошибок.

Если функция делает слишком много действий, она:

- становится трудно читаемой;
- усложняется при отладке;
- тяжело модифицируется.

2. *Изолированность* (Независимость)

Функция должна работать с теми данными, которые ей переданы, а не с внешними случайными переменными.

Это называется контролируемая область видимости данных.

Почему это важно? Если функция использует глобальные переменные:

- возрастает риск непредсказуемых ошибок;
- усложняется тестирование;
- повышается зависимость между блоками.

Большинство сложных ошибок в автоматизации возникают именно из-за неконтролируемого изменения глобальных переменных.

Изолированность делает функцию:

- безопасной;
- предсказуемой;
- повторно используемой.

3. *Повторное использование* (Reusable logic)

Одна из самых сильных сторон функции – возможность многократного вызова. Например:

- обработка 10 листов таблицы;
- проверка 100 строк;

- формирование отчёта по нескольким категориям.

Вместо копирования логики используется одна функция, вызываемая в цикле. Это уменьшает объём алгоритма и снижает вероятность ошибок.

4. *Управляемость входных данных*

Функция принимает параметры – и именно это делает её универсальной. Например, вместо: 5 блоков обработки разных листов создаётся: 1 функция обработки листа, параметр «название листа»

Это позволяет:

- быстро масштабировать процесс;
- легко добавлять новые элементы;
- избегать дублирования логики.

Чем меньше «жёстко прописанных» значений внутри функции, тем выше её гибкость.

5. *Возврат результата*

Функция может возвращать:

- логическое значение (True / False);
- число;
- строку;
- список;
- словарь;
- таблицу.

Возврат результата делает процесс управляемым.

Главный алгоритм получает конкретный итог выполнения функции и принимает решение о дальнейших действиях.

Возврат значений позволяет строить многоуровневую логику – когда одна функция управляет вызовом другой.

6. *Тестируемость*

Функцию можно протестировать отдельно от всего процесса. Это означает:

- проще находить ошибки;
- быстрее проводить отладку;
- легче проверять корректность расчётов.

В профессиональной разработке тестируемость является критерием качества архитектуры.

7. *Масштабируемость*

Если процесс увеличивается (больше листов, больше условий, больше данных), функции позволяют:

- изменить один блок без переписывания всего алгоритма;
- добавить новую функцию без разрушения структуры;
- адаптировать процесс под новые требования.

Роботы без функций плохо масштабируются. Роботы с функциями легко расширяются.

8. *Читаемость и архитектурная прозрачность*

Алгоритм, разделённый на функции, читается как последовательность шагов высокого уровня:

- получить данные;
- обработать данные;
- сформировать результат;
- записать результат.

Это делает процесс понятным не только разработчику, но и проверяющему или заказчику. Читаемость алгоритма напрямую влияет на скорость сопровождения проекта.

Структура функции в RPA

Функция – это не просто набор действий, а логически выстроенная конструкция с чёткой внутренней организацией.

Понимание её структуры позволяет проектировать устойчивые, масштабируемые и профессиональные автоматизированные процессы.

Любая функция в RPA имеет четыре ключевых элемента:

- Название
- Входные параметры
- Внутреннюю логику
- Возвращаемое значение

Рассмотрим каждый элемент подробно.

1. *Название функции*

Название – это первый уровень архитектуры. Оно должно:

- отражать назначение;
- быть конкретным;
- быть понятным без изучения внутренней логики;
- описывать действие, а не объект.

Пример хороших названий:

- `ValidateSheetName`
- `ProcessPaymentRow`
- `CalculateTotalAmount`
- `CheckDuplicateRecords`

Пример плохих названий:

- `Function1`
- `Test`
- `Data`
- `NewBlock`
- `MyProcess`

В инженерной разработке до 40% времени сопровождения уходит на понимание логики кода. Грамотные названия сокращают это время в несколько раз.

Название – это архитектурная подсказка для разработчика и проверяющего.

2. *Входные параметры*

Входные параметры – это данные, которые функция получает извне. Они делают функцию:

- универсальной;
- управляемой;
- независимой от жёстко заданных значений.

Например: Функция обработки листа не должна «знать» конкретное название листа внутри себя. Название должно передаваться параметром. Это позволяет использовать одну и ту же функцию для:

- разных листов;
- разных таблиц;
- разных наборов данных.

Почему это важно? Если значения «зашиты» внутри функции:

- её сложно изменить;
- её нельзя переиспользовать;
- возрастает риск ошибок при расширении проекта.

В профессиональной автоматизации жёстко заданные значения считаются одной из главных причин неустойчивости процессов.

3. *Внутренняя логика*

Это «сердце» функции – последовательность действий, которые она выполняет. Правильная внутренняя логика должна быть:

- последовательной;
- читаемой;
- ограниченной одной задачей;
- не перегруженной дополнительными обязанностями.

Важный принцип: Функция должна делать одно действие качественно, а не несколько действий частично.

Плохая практика:

Функция одновременно:

- читает таблицу,
- проверяет ошибки,
- формирует отчёт,
- записывает данные.

Хорошая практика: разделить это на несколько функций.

Чем длиннее функция, тем выше вероятность логической ошибки.

Функции с большим количеством логических ветвлений имеют значительно больший процент дефектов.

4. *Возвращаемое значение*

Возврат результата – это способ взаимодействия функции с главным процессом. Возврат результата позволяет:

- строить многоуровневую логику;
- принимать решения в главном алгоритме;

- контролировать успешность выполнения.

Пример логики: `Функция View_Sheet → возвращает True или False`

Главный процесс:

- если `True` – вызвать функцию обработки;
- если `False` – перейти к следующему листу.

Это создаёт управляемую архитектуру, а не хаотичную последовательность действий.

5. *Связь функции с главным процессом*

Функция не существует отдельно. Она вызывается главным алгоритмом. Правильная структура выглядит так:

- Главный процесс:
  - ✓ получает список листов;
  - ✓ вызывает функцию обработки;
  - ✓ получает результат;
  - ✓ принимает решение.
- Функция:
  - ✓ выполняет строго свою задачу;
  - ✓ возвращает результат;
  - ✓ не управляет всем процессом целиком.
- Главный процесс – координатор.
- Функции – исполнители.

Структура функции – это не просто формальность, а система, которая обеспечивает:

- модульность;
- масштабируемость;
- повторное использование;
- управляемость;
- читаемость;
- снижение количества ошибок.

Понимание структуры функции – это шаг от простого составления алгоритмов к инженерному проектированию автоматизированных решений.

Использование функций в Puzzle RPA

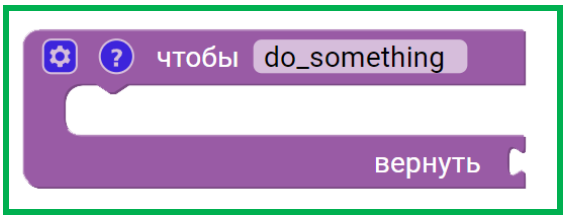
В Puzzle RPA функция – это самостоятельный логический блок, который выполняет конкретную задачу в рамках общего процесса.

Она создаётся внутри процесса как отдельный блок, может принимать входные параметры, содержать собственную логику и возвращать результат.

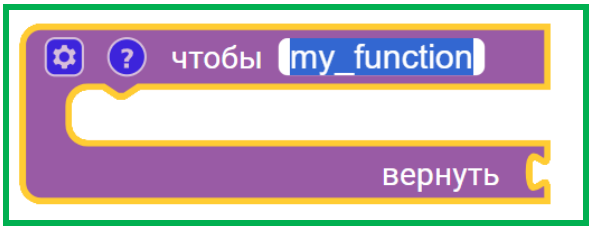
Все функции, используемые в роботах Puzzle RPA, находятся в разделе «Стандартные».

Чтобы использовать функции в Puzzle RPA требуется:

1. Создать функцию;

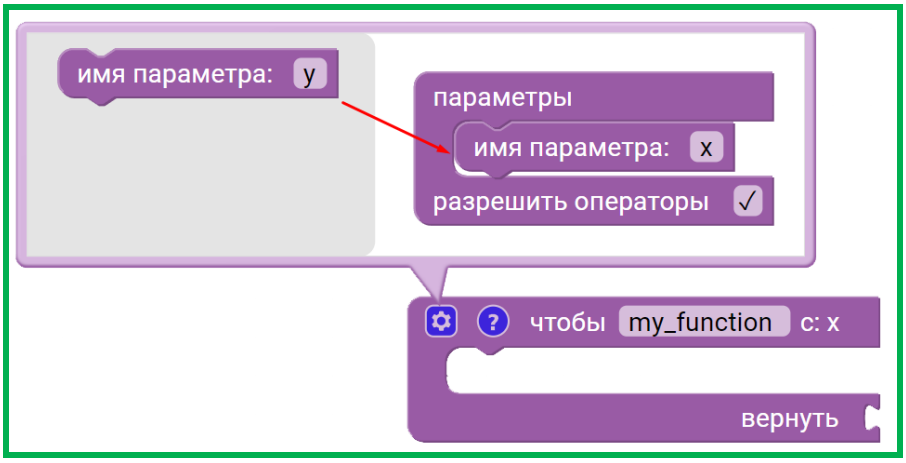


2. Указать название функции в поле ввода; В названии функции не допускаются пробелы.

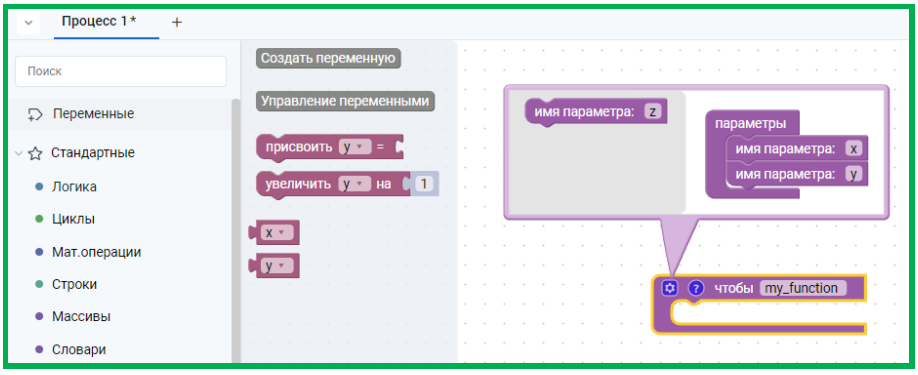


3. Добавить параметры, которые будут использоваться в алгоритме функции;

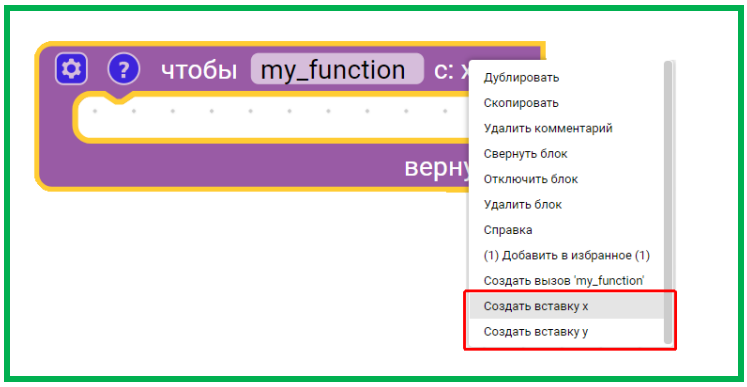
Параметры – это переменные, которые вы определяете в объявлении функции. Они служат для значений, которые будут переданы функции при её вызове. Параметры позволяют функции принимать входные данные и использовать их в своей логике.



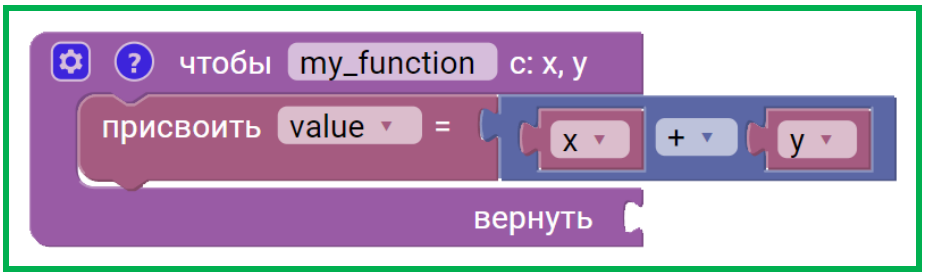
Переменные, используемые в качестве параметров функций, отображаются в списке переменных для всех процессов, независимо от того, в каком именно процессе объявлена функция.



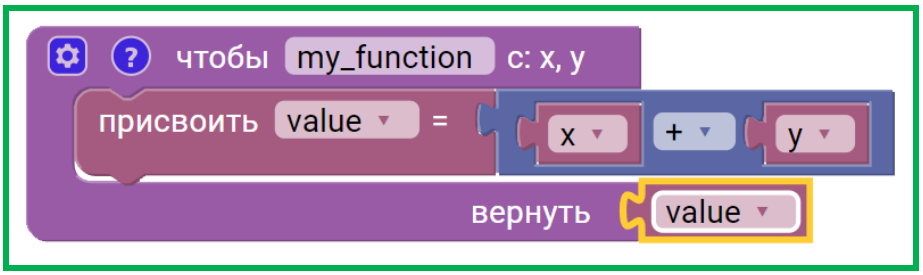
Для создания вставки переменной-параметра, необходимо открыть контекстное меню функции и кликнуть на кнопку **Создать вставку** соответствующего параметра:



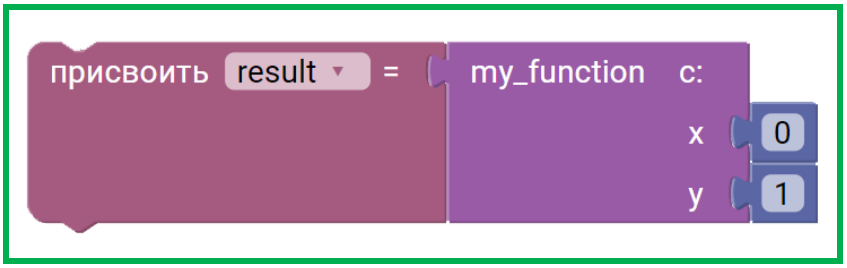
4. Собрать алгоритм внутри функции;



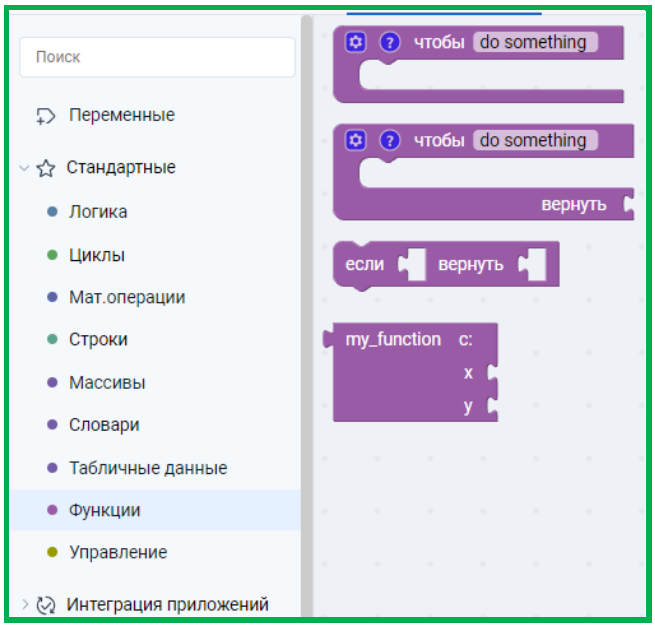
5. Указать значения, которые будут возвращены из функции, если это требуется по процессу;



6. Выполнить функцию в любом месте основного алгоритма.

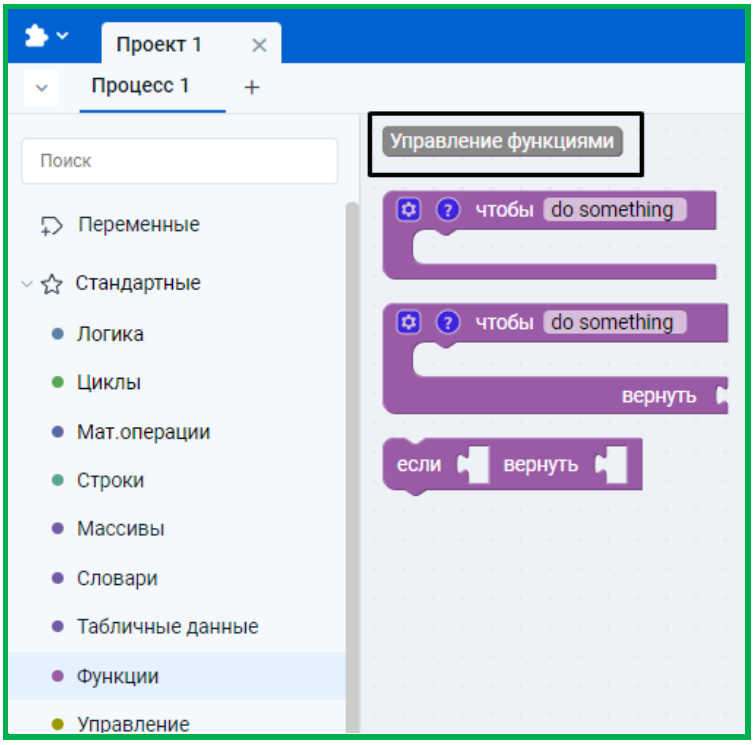


Для выполнения функции в основном алгоритме необходимо добавить блок вызова. Блок автоматически генерируется при создании функции в подкатегории “Функции”.



Функции, разработанные в дочерних процессах проекта, могут быть использованы в других процессах этого же проекта робота. При создании функции её вызов доступен во всех процессах, относящихся к одному проекту робота. Функции, объявленные в главном процессе, нельзя вызывать и использовать из дочерних процессов.

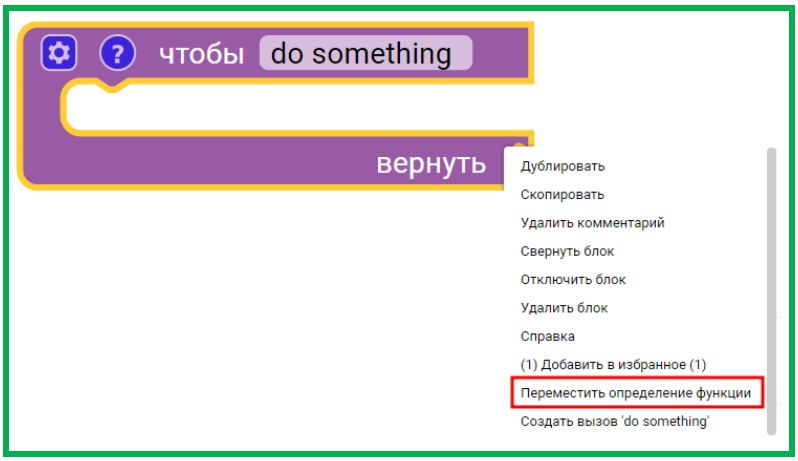
Клик на кнопку «Управление функциями» открывает встроенный инструмент студии Puzzle RPA, который отображает все функции и переменные, используемые в проекте.



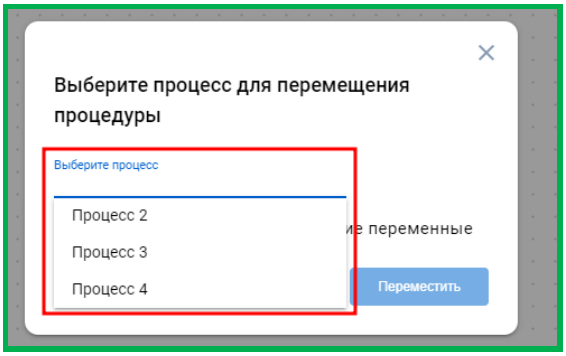
Список переменных и функций – встроенный инструмент Puzzle RPA, который отображает все функции и переменные, используемые в проекте.

Функции, разработанные в проекте, могут быть перенесены в другой процесс в данном проекте. Для этого требуется:

1. Кликнуть правой кнопкой мыши по блоку определения функции и выбрать пункт «Переместить определение функции».



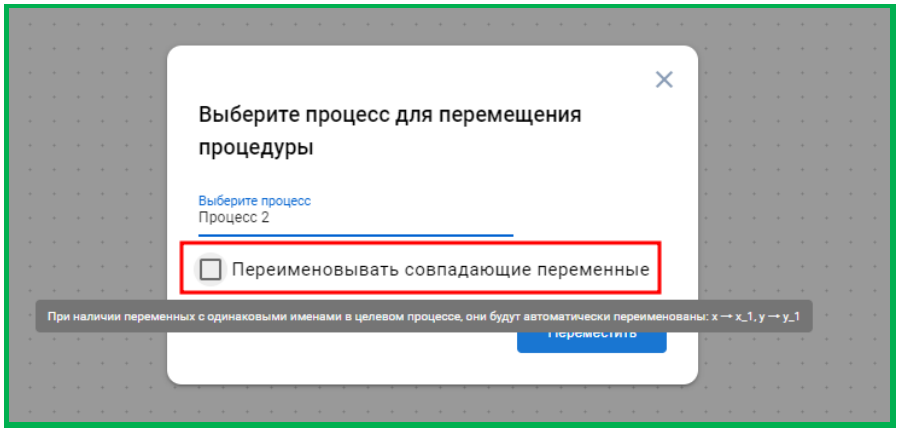
2. В открывшемся окне выбрать процесс, в который будет перенесена функция.



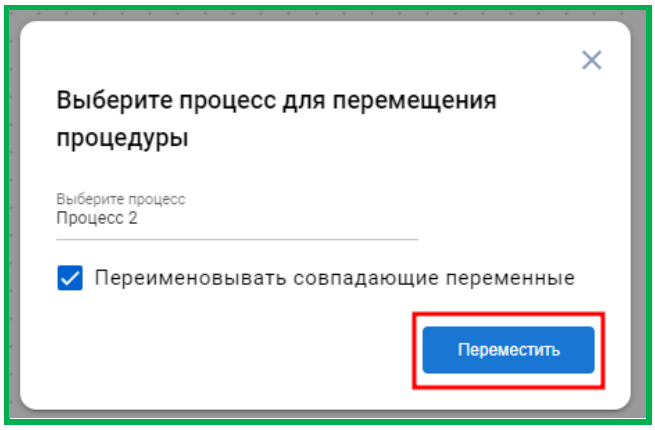
3. Определить поведение при совпадении имён переменных функции и переменных целевого процесса;

Если в целевом процессе уже существует переменная, имя которой используется в алгоритме функции, и выбрана опция «Переименовывать совпадающие переменные», то переменные в теле функции будут автоматически переименованы (например: x → x_1, y → y_1).

В случае совпадения имени параметра функции с переменной в целевом процессе, автоматическое переименование не применяется.



4. Выполнить клик на кнопку «Переместить» для подтверждения.

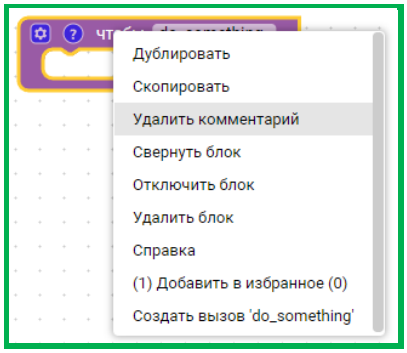


Удалить созданную функцию можно несколькими способами:

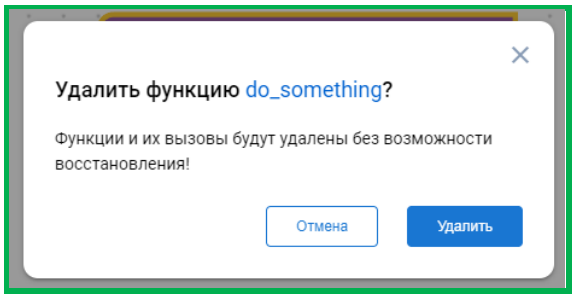
- Способ 1 – Через контекстное меню

  Для удаления через контекстное меню требуется:

  1. Выбрать функцию для удаления;
  2. Выполнить клик правой кнопкой мыши по блоку Определение функции;



  3. Кликнуть на вариант Удалить блок;
  4. Подтвердить удаление кликом на кнопку Удалить.



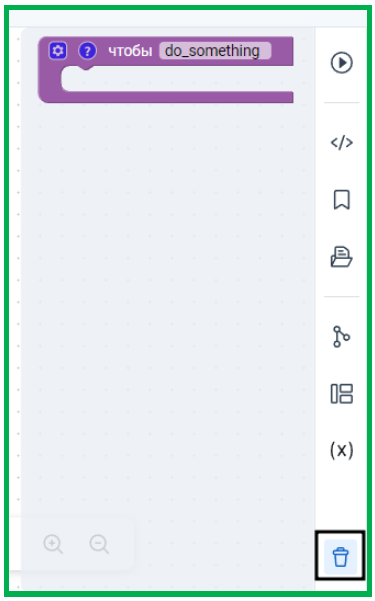
- Способ 2 – Через форму управления функциями

Клик на кнопку «Управление функциями» открывает встроенный инструмент студии Puzzle RPA, который отображает все функции и переменные, используемые в проекте. Этот инструмент позволяет выполнить удаление функций.

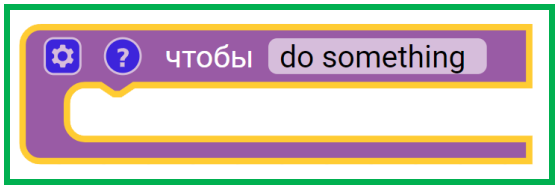
Невозможно:

- Отменить удаление функций комбинаций клавиш **Ctrl + Z**;
- Вырезать объявление функции комбинаций клавиш **Ctrl + X**.

Восстановить удаленную функцию можно только из корзины:



1. Блок Определение функции

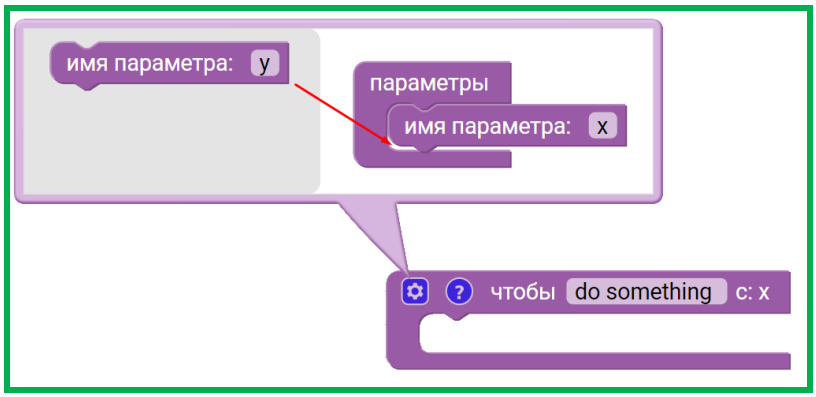


Блок создает процедуру, которая выполняет определенные действия, но не возвращает никакого значения.

Чтобы открыть форму управления параметрами функции, требуется кликнуть на кнопку:



Для добавления нового параметра в функцию необходимо перетащить его в область параметров в интерфейсе управления параметрами функции. При необходимости можно также изменить имя добавленного параметра.



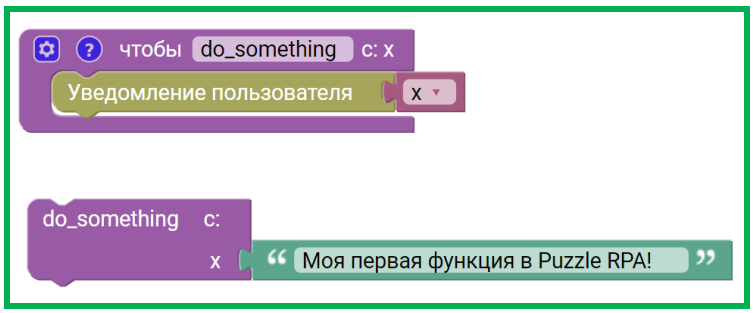
Для выполнения функции в основном алгоритме необходимо добавить блок вызова.



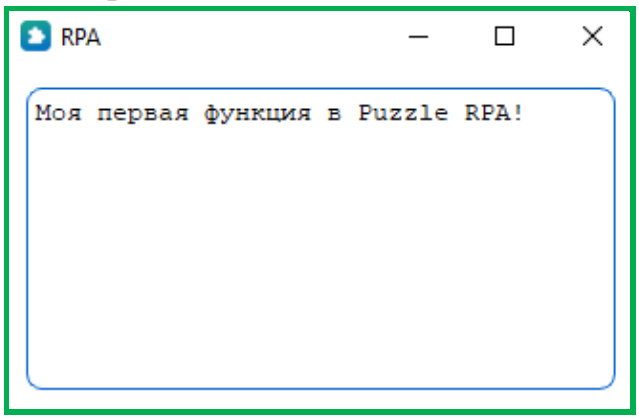
Блок автоматически генерируется при создании функции в подкатегории «Функции».

Пример использования

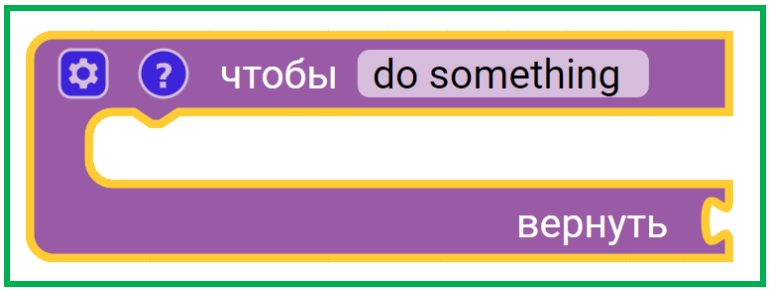
В данном примере создается функция `do_something`, которая принимает параметр `x`. Значение параметра выводится на экран через блок “Уведомление пользователя”. В качестве аргумента функции передается строка “Моя первая функция в Puzzle RPA!”.



Вывод результата на экран:



2. Блок Определение функции и возврат значения

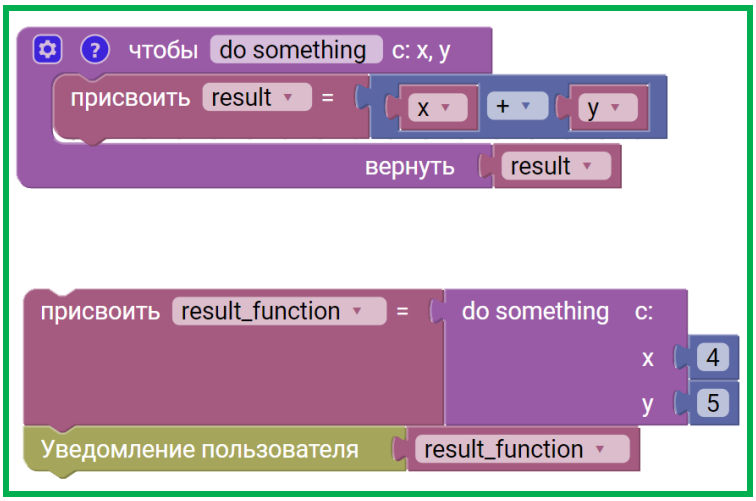


Блок создает процедуру, которая выполняет определенные действия и возвращает результат выполнения.

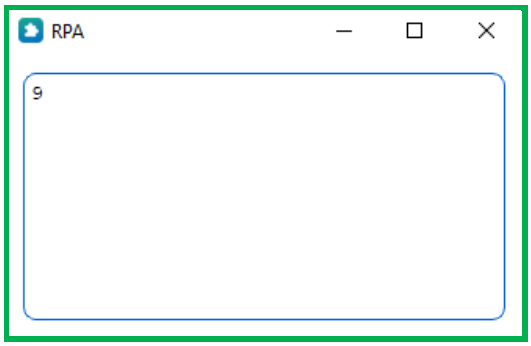
Пример использования:

В данном примере создается функция `do_something`, которая принимает параметр `x` и `y`. Внутри функции выполняется сложение двух принимаемых параметров. Результат возвращается из функции.

При вызове функции `do_something` в качестве аргументов передаются 2 числа: 4 и 5.



Вывод результата на экран:



3. Блок Если–вернуть

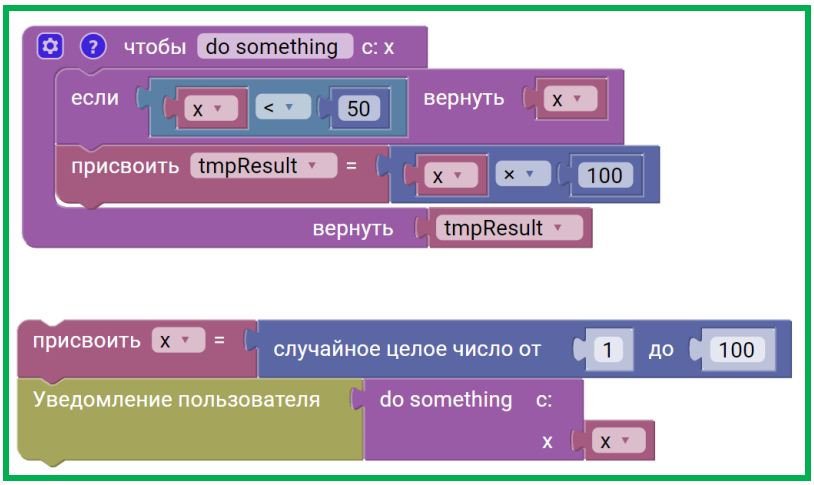


Блок помещается внутрь функции. Если условие внутри блока истина, то возвращается указанное значение.

Пример использования

В данном примере блок возвращает число из функции без изменения, если случайное число меньше 50:

- Генерируется случайное число от 1 до 100;
- Вызывается функция, которая проверяет, если число менее 50, то возвращается число без изменения, в ином случае сгенерированное число увеличивается в 100 раз.



Возвращенное число без изменения:

КОНТРОЛЬНЫЕ ЗАДАНИЯ

Контрольное задание 1

Этап 1: Проверка наличия подтверждающих документов

Ситуация

На первом этапе система должна проверить, приложил ли сотрудник подтверждающие документы (чеки) к отчёту. Отчёты без чеков не могут быть приняты к обработке и требуют дополнения.

Бухгалтерии необходимо получить отдельный файл со статусом комплектности документов по каждому отчёту.

Требуется:

1. **Считать JSON-файл (6 отчётов).**
2. **Организовать цикл обработки списка словарей.**
3. **Создать функцию с возвратом значения.**

Функция CheckDocumentsProvided

Тип: Определение функции и возврат значения Параметр:

- **report_dict**

Логика:

- **Получить значение receipts_total из словаря.**
- **Если receipts_total = 0 → вернуть "Документы отсутствуют".**
- **Если receipts_total > 0 → вернуть "Документы предоставлены".**

Главный процесс должен:

1. **Создать пустой список documents_check_list.**
2. **В цикле:**
    - **Вызвать CheckDocumentsProvided;**
    - **Сформировать словарь:**

```
{
 "employee_name": "Орлов О.О.",
 "documents_status": "Документы предоставлены"
}
```

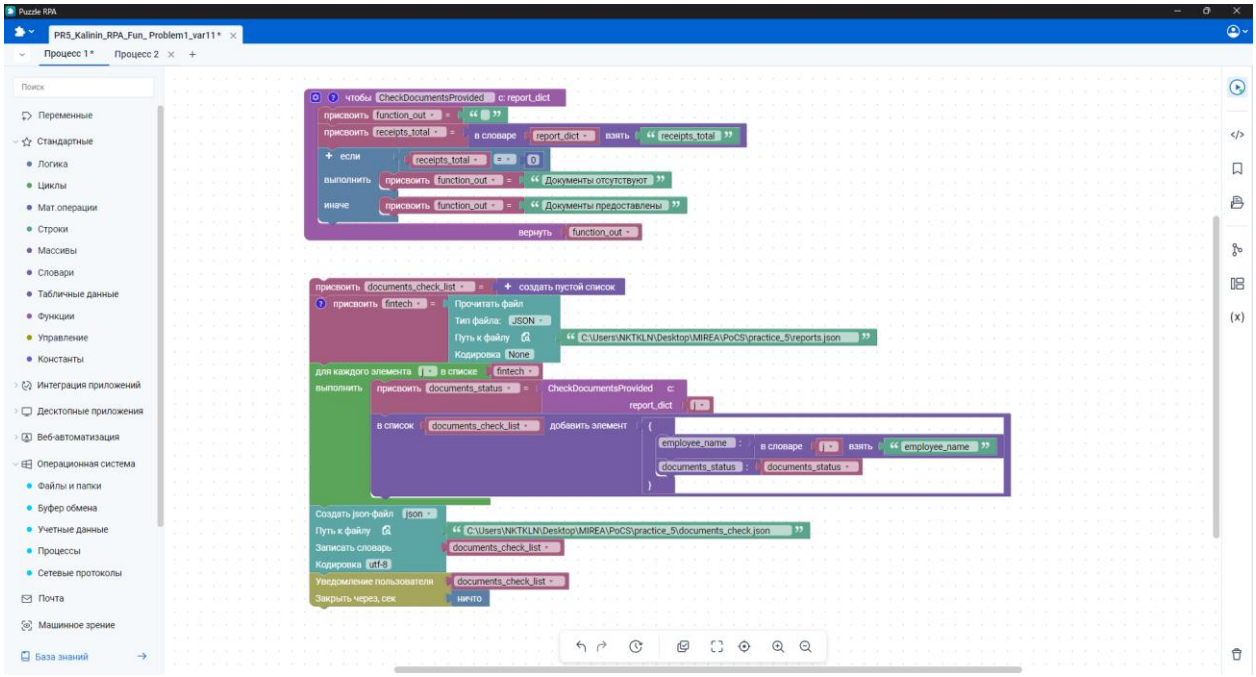
- **Добавить словарь в documents_check_list.**

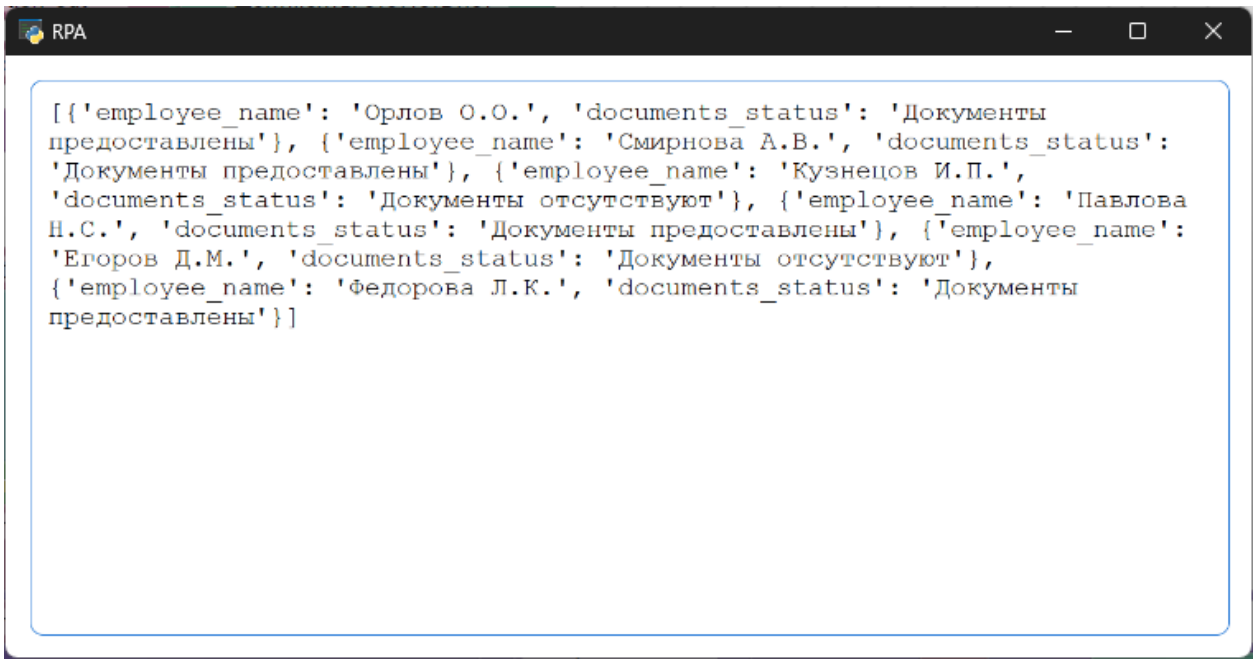
3. **Сохранить documents_check_list в файл documents_check.json.**
4. **Вывести итоговый список на экран.**

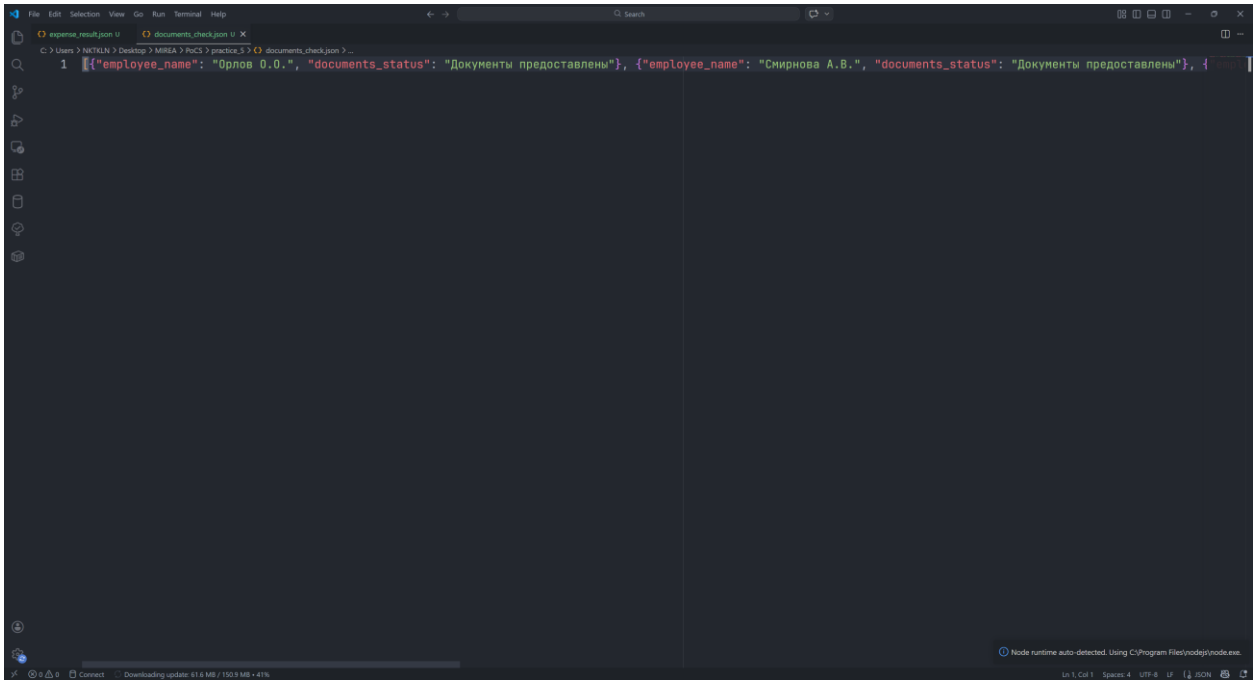
Архитектурные требования

- **Функция создаётся через блок «Определение функции и возврат значения».**
- **Функция выполняет только одну задачу — проверку наличия чеков.**
- **Запрещено использовать глобальные переменные.**
- **Главный процесс не содержит логики проверки**

Успешное выполнение кода процесса







Алгоритм решения: считать JSON-файл reports.json, содержащий 6 авансовых отчётов в формате списка словарей. Далее организовать цикл обработки, в котором на каждой итерации берётся один словарь отчёта сотрудника и передаётся в пользовательскую функцию CheckDocumentsProvided(report_dict). Функция получает значение поля receipts_total и выполняет только одну задачу — проверку наличия подтверждающих документов: если сумма чеков равна 0, функция возвращает строку "Документы отсутствуют", если сумма больше 0 — "Документы предоставлены". В главном процессе создаётся пустой список documents_check_list, в который по результатам каждого вызова функции добавляется новый словарь вида employee_name и documents_status. После завершения цикла итоговый список сохраняется в файл documents_check.json, а затем выводится на экран для передачи в бухгалтерию.

Контрольное задание 2

Этап 2: Расчёт итоговой суммы и статуса расчётов

Ситуация

После проверки документов руководство решило автоматизировать финальный расчёт по командировке.

Теперь требуется рассчитать общую сумму расходов (суточные + чеки), сравнить её с выданным авансом и определить статус расчётов (долг сотрудника, долг компании или погашено).

Бухгалтерии нужен структурированный список отчётов с итоговыми суммами для проведения платежей или требования возврата.

Требуется:

1. **Использовать исходный JSON-файл из 6 отчётов.**
2. **Организовать цикл обработки списка.**
3. **Создать три функции с возвратом значения.**

1️⃣ Функция CalculateTotalExpense

Параметр:

- **report_dict**

Логика:

- **Получить trip_days, daily_allowance и receipts_total.**
- **Рассчитать сумму суточных: trip_days * daily_allowance.**
- **Рассчитать общую сумму: суточные + receipts_total.**
- **Вернуть число.**

2️⃣ Функция DetermineSettlementStatus

Параметры:

- **total_expense**
- **advance_given**

Логика:

- **Если total_expense = advance_given → вернуть "Расчёты погашены".**
- **Если total_expense > advance_given → вернуть "Доплата сотруднику".**
- **Если total_expense < advance_given → вернуть "Возврат в кассу".**

3️⃣ Функция BuildExpenseRecord

Параметры:

- **employee_name**
- **settlement_status**
- **total_expense**
- **advance_given**

Логика:

- **Рассчитать разницу: difference_amount = abs(total_expense - advance_given).**
- **Создать и вернуть словарь:**

{

"employee_name": "Орлов О.О.",

"settlement_status": "Доплата сотруднику",

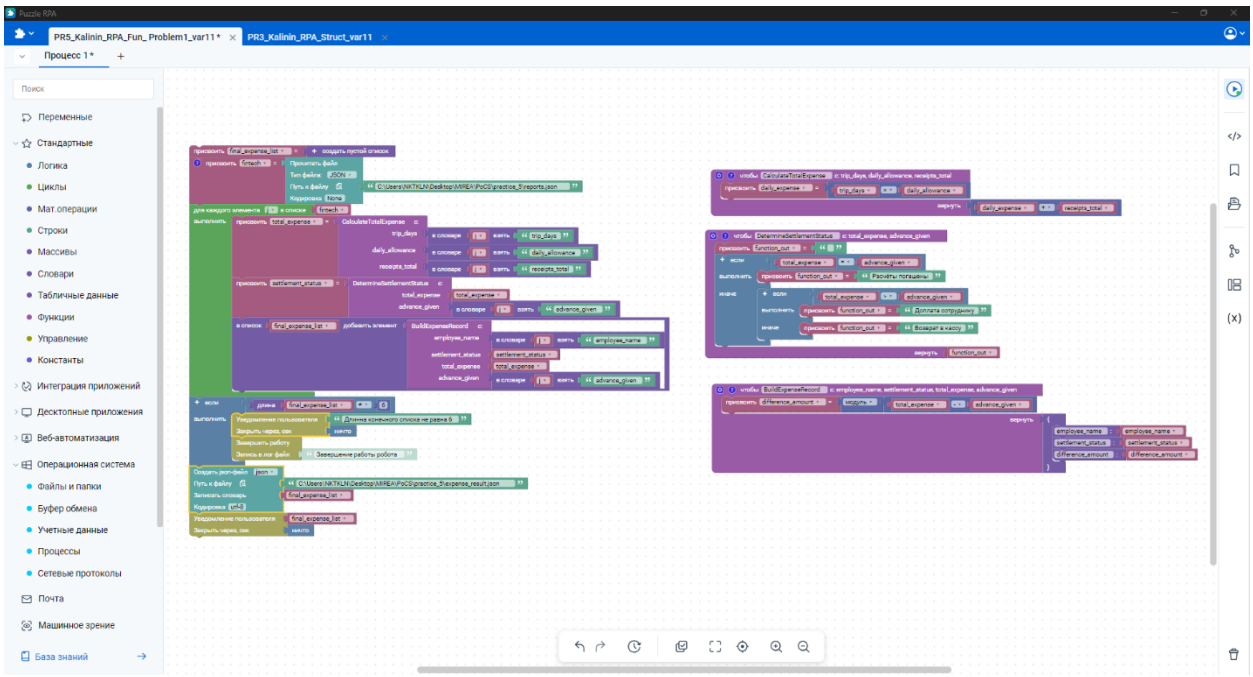
"difference_amount": 5000.00

}

Главный процесс должен:

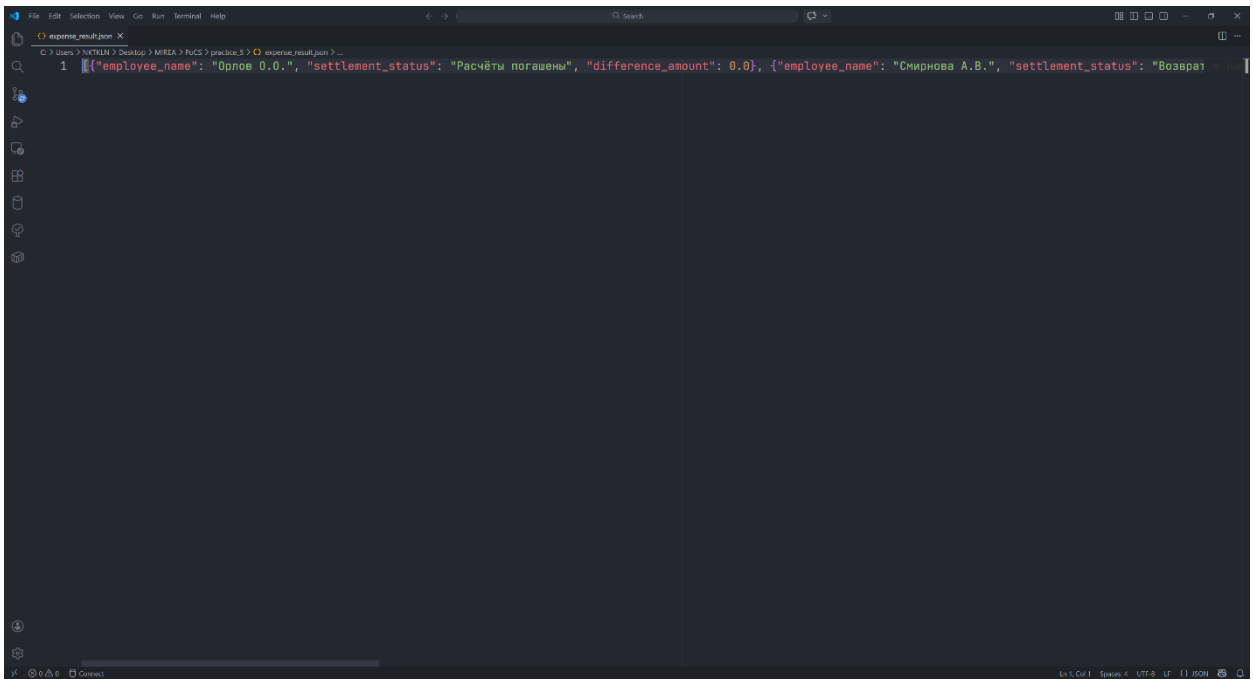
1. **Создать пустой список final_expense_list.**
2. **В цикле для каждого отчёта:**
    - **Вызвать CalculateTotalExpense;**
    - **Вызвать DetermineSettlementStatus (передать сумму расходов и аванс);**
    - **Вызвать BuildExpenseRecord;**
    - **Добавить возвращённый словарь в final_expense_list.**
3. **Сохранить final_expense_list в файл expense_result.json.**
4. **Проверить, что итоговый файл содержит 6 записей.**
5. **Вывести итоговый список на экран.**

Успешное выполнение кода процесса



RPA

```
[{'employee_name': 'Орлов О.О.', 'settlement_status': 'Расчёты погашены',
'difference_amount': 0.0}, {'employee_name': 'Смирнова А.В.',
'settlement_status': 'Возврат в кассу', 'difference_amount': 1600.0},
{'employee_name': 'Кузнецов И.П.', 'settlement_status': 'Возврат в
кассу', 'difference_amount': 7600.0}, {'employee_name': 'Павлова Н.С.',
'settlement_status': 'Возврат в кассу', 'difference_amount': 800.0},
{'employee_name': 'Егоров Д.М.', 'settlement_status': 'Возврат в кассу',
'difference_amount': 19000.0}, {'employee_name': 'Федорова Л.К.',
'settlement_status': 'Расчёты погашены', 'difference_amount': 0.0}]
```



Алгоритм решения: считать исходный JSON-файл reports.json, содержащий 6 авансовых отчётов в формате списка словарей. Далее организовать цикл обработки списка, в котором на каждой итерации один отчёт последовательно передаётся в три пользовательские функции. Сначала функция CalculateTotalExpense(report_dict) получает из словаря количество дней командировки, размер суточных и сумму чеков, рассчитывает сумму суточных как произведение trip_days * daily_allowance, затем определяет общую сумму расходов как сумму суточных и receipts_total, после чего возвращает рассчитанное числовое значение. Затем функция DetermineSettlementStatus(total_expense, advance_given) сравнивает общую сумму расходов с выданным авансом и возвращает один из статусов расчётов: "Расчёты погашены", "Доплата сотруднику" или "Возврат в кассу". После этого функция BuildExpenseRecord(employee_name, settlement_status, total_expense, advance_given) вычисляет абсолютную разницу между общей суммой расходов и авансом, формирует новый словарь с фамилией сотрудника, статусом расчётов и суммой разницы, и возвращает его в главный процесс. В главном процессе создаётся пустой список final_expense_list, в который на каждой итерации добавляется словарь, возвращённый функцией BuildExpenseRecord. После завершения обработки всех 6 отчётов итоговый список сохраняется в файл expense_result.json, затем выполняется проверка количества записей в итоговом файле, и готовый список выводится на экран для дальнейшей работы бухгалтерии.

Ссылка на репозиторий: https://github.com/NKTKLN/mirea-puzzle-rpa/tree/main/practice_5

Ссылка на чат DeepSeek: https://chat.deepseek.com/share/n6u2hqadxc59os3wzy